

WEEK 4: CUSTOMER ORDER HISTORY AND PURCHASE REPORTING

Scenario:

An online store wants to generate reports about customer orders and product sales. The system stores customer details, order information, and product order details. These reports help to view customer order history, product sales, and customer purchase information.

1. Design Order and Order Details Tables:

Order Table:

```
SQL> CREATE TABLE Orders (  
2     Order_ID VARCHAR2(7) PRIMARY KEY,  
3     Customer_ID VARCHAR2(7),  
4     Order_Date DATE,  
5     Total_Amount NUMBER(10,2),  
6     Status VARCHAR2(20)  
7 );
```

Table created.

Order Details Table:

```
SQL> CREATE TABLE Order_Details (  
2     Order_Detail_ID VARCHAR2(7) PRIMARY KEY,  
3     Order_ID VARCHAR2(7),  
4     Product_ID VARCHAR2(7),  
5     Quantity NUMBER,  
6     Price NUMBER(10,2),  
7     FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID)  
8 );
```

Table created.

2. Define Relationships and Constraints

Relationships:

The Order_ID in the Orders table connects the order with its corresponding order details. To create this relationship, we need to identify the keys that uniquely identify records and connect the two tables.

PRIMARY KEY:

Order_ID uniquely identifies each order, while Order_Detail_ID uniquely identifies each order detail.

FOREIGN KEY:

Order_ID in the Order_Details table connects it with the Orders table. This relationship helps maintain accurate order information and details.

NOT NULL:

Ensures that a column must always contain a value. For example, **Order Date** and **Order Status** should always be provided.

CHECK:

Ensures that the entered values follow a specific condition. For example, **Quantity** and **Price** should be greater than zero.

UNIQUE:

Ensures that duplicate values are not allowed in a column. For example, each **Order_Detail_ID** should be unique. Defining Relationship and Creating Constraints for Order and Order Details

Primary Key:

```
SQL> ALTER TABLE Orders
  2  ADD CONSTRAINT PK_Orders PRIMARY KEY (Order_ID);

Table altered.
```

```
SQL> ALTER TABLE Order_Details
  2  ADD CONSTRAINT FK_OrderDetails_Orders
  3  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

Table altered.
```

Creating Relationship using **FOREIGN KEY**:

```
SQL> ALTER TABLE Order_Details
  2  ADD CONSTRAINT FK_OrderDetails_Orders
  3  FOREIGN KEY (Order_ID) REFERENCES Orders(Order_ID);

Table altered.
```

```
SQL> ALTER TABLE Order_Details
  2  ADD CONSTRAINT FK_OrderDetails_Product
  3  FOREIGN KEY (Product_ID) REFERENCES Product(Product_ID);

Table altered.
```

Adding Constraints: NOT NULL

```
SQL> ALTER TABLE Orders  
2 MODIFY Order_Date DATE NOT NULL;
```

Table altered.

```
SQL> ALTER TABLE Order_Details  
2 MODIFY Quantity NUMBER NOT NULL;
```

Table altered.

CHECK:

```
SQL> ALTER TABLE Orders  
2 ADD CONSTRAINT CHK_Total_Amount CHECK (Total_Amount >= 0);
```

Table altered.

```
SQL> ALTER TABLE Order_Details  
2 ADD CONSTRAINT CHK_Quantity CHECK (Quantity > 0);
```

Table altered.

UNIQUE:

```
SQL> ALTER TABLE Orders  
2 ADD CONSTRAINT UQ_Tracking_Number UNIQUE (Tracking_Number);
```

Table altered.

```
SQL> ALTER TABLE Order_Details  
2 ADD CONSTRAINT UQ_Detail_Ref_Code UNIQUE (Detail_Ref_Code);
```

Table altered.

3. Insert Values: Order Table:

```
SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (101, 1, TO_DATE('2026-01-10', 'YYYY-MM-DD'), 25000.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (102, 2, TO_DATE('2026-01-12', 'YYYY-MM-DD'), 1500.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (103, 3, TO_DATE('2026-01-15', 'YYYY-MM-DD'), 3200.00, 'Pending');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (104, 1, TO_DATE('2026-01-18', 'YYYY-MM-DD'), 45000.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (105, 4, TO_DATE('2026-01-20', 'YYYY-MM-DD'), 800.00, 'Cancelled');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (106, 5, TO_DATE('2026-01-22', 'YYYY-MM-DD'), 12000.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (107, 2, TO_DATE('2026-01-25', 'YYYY-MM-DD'), 2100.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (108, 6, TO_DATE('2026-01-28', 'YYYY-MM-DD'), 5500.00, 'Shipped');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (109, 3, TO_DATE('2026-02-01', 'YYYY-MM-DD'), 18000.00, 'Completed');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (110, 7, TO_DATE('2026-02-05', 'YYYY-MM-DD'), 950.00, 'Pending');
1 row created.

SQL> INSERT INTO Orders (Order_ID, Customer_ID, Order_Date, Total_Amount, Status) VALUES (111, 8, TO_DATE('2026-02-10', 'YYYY-MM-DD'), 3400.00, 'Completed');
1 row created.
```

Order Details Table:

```
SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (1, 101, 201, 1, 25000.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (2, 102, 202, 2, 750.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (3, 103, 203, 1, 3200.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (4, 104, 201, 1, 25000.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (5, 104, 204, 1, 20000.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (6, 105, 205, 1, 800.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (7, 106, 206, 2, 6000.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (8, 107, 202, 1, 750.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (9, 108, 207, 1, 5500.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (10, 109, 208, 3, 6000.00);

1 row created.

SQL> INSERT INTO Order_Details (Order_Detail_ID, Order_ID, Product_ID, Quantity, Price)
2 VALUES (11, 110, 205, 1, 950.00);

1 row created.
```

After Insertion Order Table:

```
SQL> SELECT * FROM Orders;
```

ORDER_I	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	STATUS
TRACKING_NUMBER				

101	1	10-JAN-26	25000	Completed
-----	---	-----------	-------	-----------

102	2	12-JAN-26	1500	Completed
-----	---	-----------	------	-----------

103	3	15-JAN-26	3200	Pending
-----	---	-----------	------	---------

ORDER_I	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	STATUS
TRACKING_NUMBER				

104	1	18-JAN-26	45000	Completed
-----	---	-----------	-------	-----------

105	4	20-JAN-26	800	Cancelled
-----	---	-----------	-----	-----------

106	5	22-JAN-26	12000	Completed
-----	---	-----------	-------	-----------

ORDER_I	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	STATUS
TRACKING_NUMBER				

107	2	25-JAN-26	2100	Completed
-----	---	-----------	------	-----------

108	6	28-JAN-26	5500	Shipped
-----	---	-----------	------	---------

109	3	01-FEB-26	18000	Completed
-----	---	-----------	-------	-----------

ORDER_I	CUSTOME	ORDER_DAT	TOTAL_AMOUNT	STATUS
TRACKING_NUMBER				

110	7	05-FEB-26	950	Pending
-----	---	-----------	-----	---------

111	8	10-FEB-26	3400	Completed
-----	---	-----------	------	-----------

```
11 rows selected.
```

Order Details Table:

```
SQL> SELECT * FROM Order_Details;
```

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
-----	-----	-----	-----	-----
DETAIL_REF_CODE				

1	101	201	1	25000
---	-----	-----	---	-------

2	102	202	2	750
---	-----	-----	---	-----

3	103	203	1	3200
---	-----	-----	---	------

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
-----	-----	-----	-----	-----
DETAIL_REF_CODE				

4	104	201	1	25000
---	-----	-----	---	-------

5	104	204	1	20000
---	-----	-----	---	-------

6	105	205	1	800
---	-----	-----	---	-----

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
-----	-----	-----	-----	-----
DETAIL_REF_CODE				

7	106	206	2	6000
---	-----	-----	---	------

8	107	202	1	750
---	-----	-----	---	-----

9	108	207	1	5500
---	-----	-----	---	------

ORDER_D	ORDER_I	PRODUCT	QUANTITY	PRICE
-----	-----	-----	-----	-----
DETAIL_REF_CODE				

10	109	208	3	6000
----	-----	-----	---	------

11	110	205	1	950
----	-----	-----	---	-----

```
11 rows selected.
```

Report 1: Customer Order History

```
SQL> SELECT
  2     c.First_Name || ' ' || c.Last_Name AS Customer_Name,
  3     o.Order_ID,
  4     o.Order_Date,
  5     o.Total_Amount,
  6     o.Status
  7 FROM Orders o
  8 LEFT JOIN Customer c ON TO_CHAR(c.Customer_ID) = TO_CHAR(o.Customer_ID)
  9 WHERE ROWNUM = 1;
```

CUSTOMER_NAME	ORDER_I	ORDER_DAT	TOTAL_AMOUNT
-----	-----	-----	-----
STATUS			

Completed	104	18-JAN-26	45000

Report 2: Product-wise Order Report

```
SQL> SELECT * FROM (
  2     SELECT
  3         od.Product_ID,
  4         COUNT(od.Order_ID) AS Times_Ordered,
  5         SUM(od.Quantity) AS Total_Quantity_Sold
  6     FROM Order_Details od
  7     GROUP BY od.Product_ID
  8 ) WHERE ROWNUM = 1;
```

PRODUCT	TIMES_ORDERED	TOTAL_QUANTITY_SOLD
-----	-----	-----
201	2	2

Report 3: Customer Purchase Analysis

1. Customer with maximum orders

```
SQL> SELECT * FROM (
  2     SELECT
  3         o.Customer_ID,
  4         NVL(MAX(c.First_Name || ' ' || c.Last_Name), 'Customer ' || o.Customer_ID) AS Customer_Name,
  5         COUNT(o.Order_ID) AS Total_Orders
  6     FROM Orders o
  7     LEFT JOIN Customer c ON TO_CHAR(c.Customer_ID) = TO_CHAR(o.Customer_ID)
  8     GROUP BY o.Customer_ID
  9     ORDER BY COUNT(o.Order_ID) DESC
 10 ) WHERE ROWNUM = 1;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_ORDERS
-----	-----	-----
1		2

2. Total Spending by Customer

```
SQL> SELECT * FROM (
  2     SELECT
  3         o.Customer_ID,
  4         NVL(MAX(c.First_Name || ' ' || c.Last_Name), 'Customer ' || o.Customer_ID) AS Customer_Name,
  5         SUM(o.Total_Amount) AS Total_Spending
  6     FROM Orders o
  7     LEFT JOIN Customer c ON TO_CHAR(c.Customer_ID) = TO_CHAR(o.Customer_ID)
  8     GROUP BY o.Customer_ID
  9 ) WHERE ROWNUM = 1;
```

CUSTOMER_ID	CUSTOMER_NAME	TOTAL_SPENDING
-----	-----	-----
1		70000

3. Average Order Value

```
SQL> SELECT
  2     AVG(Total_Amount) AS Average_Order_Value
  3   FROM Orders;

AVERAGE_ORDER_VALUE
-----
      10677.2727
```